

Depth Sensor Noise (Isaac Sim 6.0.1)

wrist camera에서 얻는 depth/point cloud에 실제 센서에 가까운 노이즈를 넣는 두 가지 경로를 정리한다. 이 문서의 모든 수치는 이 워크스테이션(RTX 5080, Isaac Sim 6.0.1-rc.7)에서 직접 실행해 얻은 값이다.

경로	무엇을 모사하나	언제 쓰나
RTX stereo depth sensor	스테레오 시차 계산 자체 — 구멍, 최소 거리, 텍스처 의존성	실제 D455 거동을 재현할 때
Replicator augmentation	임의의 수식 노이즈 (축방향 $\sigma \propto z^2$, dropout 등)	논문 노이즈 모델을 그대로 재현할 때

두 경로는 배타적이지 않다. 구조적 결함은 stereo sensor가 만들고, 그 위에 augmentation으로 추가 노이즈를 얻을 수 있다.

1. RTX stereo depth sensor

`Camera_Pseudo_Depth` 를 그냥 읽으면 이상화된 핀홀 depth라 노이즈도 구멍도 없다. Isaac Sim 6.0.1은 렌더러 후처리로 스테레오 시차를 계산하는 depth sensor를 제공하며, 노이즈는 disparity 단계에 들어간다.

API

`isaacsim.sensors.camera.SingleViewDepthSensor` 는 **6.0.0부터 deprecated**다 (`extsDeprecated/` 에 있고 확장 toml에 deprecation 경고가 박혀 있다). 새 코드는 `isaacsim.sensors.experimental.rtx` 를 쓴다.

```
import numpy as np
from isaacsim.sensors.experimental.rtx import RtxCamera, SingleViewDepthCameraSensor

# 카메라 prim에 OmniSensorAPI 스키마가 필요하다. 기존 prim 경로에 대해서도
# create()를 호출하면 스키마를 적용해 준다. 스키마 없는 prim을 그냥 감싸면
# "Prim at ... does not have the 'OmniSensorAPI' schema" 로 죽는다.
cam = RtxCamera.create("/World/cam", positions=..., orientations=...)

sensor = SingleViewDepthCameraSensor(
    cam,
    resolution=(480, 640),          # (height, width) — OpenCV/NumPy 규약. 기존 Camera와 반대다
    annotators=["depth_sensor_distance", "depth_sensor_point_cloud_position"],
)

data, info = sensor.get_data("depth_sensor_distance")  # warp array
depth = data.numpy()                                  # (H, W, 1)
```

사용 가능한 depth sensor 어노테이터는 `depth_sensor_distance`, `depth_sensor_point_cloud_position`, `depth_sensor_point_cloud_color`, `depth_sensor_imager` 네 가지다. 표준 어노테이터 (`distance_to_image_plane`, `pointcloud` 등)도 같은 객체에서 함께 받을 수 있어 ground truth 비교에 쓴다.

검증된 기본값

`world.reset()` + `app_utils.play(commit=True)` 이후 읽은 값이다.

파라미터	기본값	세터
noise mean / sigma	0.25 / 0.25 px	<code>set_sensor_noise_parameters()</code>
noise downscale	1.0 px	<code>set_sensor_disparity_noise_downscale()</code>
baseline	55.0 mm	<code>set_sensor_baseline()</code>
focal length	897.0 px	<code>set_sensor_focal_length()</code>
max disparity	110.0 px	<code>set_sensor_maximum_disparity()</code>
distance cutoffs	(0.5, 1e7) m	<code>set_sensor_distance_cutoffs()</code>
disparity confidence	0.70	<code>set_sensor_disparity_confidence()</code>
post processing	True	<code>set_enabled_post_processing()</code>

노이즈는 **기본으로 이미 켜져 있다**. mean은 양자화 폭, sigma는 그 안에서의 산포다.

이상적 depth(`distance_to_image_plane`) 대비 실측 오차는 평균 거리 2.82 m에서 bias `-0.033 m`, std `0.236 m` 였다.

함정 1: 최소 측정 거리 — wrist camera에 직접 영향

스테레오이므로 최소 측정 거리는 파라미터에서 유도된다.

```
min_range = focal_px × baseline_m / max_disparity_px
           = 897 × 0.055 / 110 = 0.4485 m
```

여기에 `minDistance=0.5` 컷오프까지 겹친다. 즉 **기본값으로는 0.5 m보다 가까운 것의 depth가 전혀 안 나온다**. 실측에서 카메라를 0.4 m 거리에 두자 307,200 픽셀 중 유효 픽셀이 5,961개뿐이었고 유효 depth가 정확히 1.0 m 부터 시작했다.

grasp 접근 단계의 wrist camera-물체 거리는 0.1~0.4 m라 이 구간이 통째로 비는데, 이건 시뮬레이터 결함이 아니라 실제 D455의 물리적 한계와 같다. 근접에서 depth가 필요하다 다음을 조정한다.

```
sensor.set_sensor_maximum_disparity(255.0)           # min_range 0.4485 -> 0.1935 m
sensor.set_sensor_distance_cutoffs(minimum_distance=0.15, maximum_distance=5.0)
```

현재 `source/sim/camera.py` 가 쓰는 이상화 depth 경로에는 이 제약이 없다. 시물에서만 되고 실기에서 안 되는 상황을 피하려면 실기 검증 전에 이 sensor로 한 번 돌려봐야 한다.

합정 2: 텍스처 없는 표면에서는 depth가 안 나온다

시야를 가득 채운 무텍스처 평면 벽(1.0 m)을 향하게 했더니 유효 픽셀이 **0개**였다. `distance_to_image_plane` 은 307,200개 전부 정상이었다. 스테레오는 대응점을 찾아야 하므로 특징이 없으면 실패한다 — 실제 RealSense가 IR 패턴 프로젝터를 다는 이유와 같다. 벤치마크 장면을 만들 때 바닥 그리드나 물체 텍스처가 있어야 한다.

노이즈를 키우면 값이 흔들리기보다 **신뢰도 미달로 픽셀이 버려진다**.

noise (mean, sigma)	유효 픽셀
0.0 / 0.0	50,996 (16.6%)
0.25 / 0.25	36,412 (11.9%)
0.25 / 1.0	32,252 (10.5%)
0.5 / 2.0	30,216 (9.8%)

합정 3: D455 프리셋은 아직 그대로 쓰지 말 것

`SUPPORTED_CAMERA_CONFIGS` 에 실제 센서 프리셋이 들어 있고 RealSense도 있다 (`/Isaac/Sensors/RealSense/D455/rsd455.usd` , `is_depth_sensor: True`). D457, D555, Orbbec Gemini, Luxonis OAK 계열도 같이 있다.

```
cam = RtxCamera.create("/World/cam", config="/Isaac/Sensors/RealSense/D455/rsd455.usd", ...)
```

다만 이 경로에서 두 가지 문제를 확인했다.

문제 1 — 에셋이 중력으로 낙하한다. `rsd455.usd`는 `RigidBodyAPI` 가 켜진 채 배포되어 물리 조인트 없이 참조하면 자유낙하한다. 관측된 거리가 5.98 → 17.4 → 50 m로 계속 증가했다. `source/sim/camera.py` 가 이미 같은 이유로 이 API를 끄고 있다.

```
for prim in Usd.PrimRange(get_prim_at_path(cam_path)):  
    if prim.HasAPI(UsdPhysics.RigidBodyAPI):  
        UsdPhysics.RigidBodyAPI(prim).GetRigidBodyEnabledAttr().Set(False)
```

문제 2 — 낙하를 막아도 depth 값의 스케일이 맞지 않는다. 같은 카메라의 `distance_to_image_plane` 중앙값 대비 `depth_sensor_distance` 중앙값이 약 1.59×10^4 배로 나왔다(단위 환산으로 설명되지 않는 값). 원인 미확인.

따라서 지금은 **plain camera prim + 파라미터 수동 설정** 경로가 안전하다. 이쪽은 1.92~9.86 m 구간에서 정상 동작을 확인했다. D455 프리셋은 원인을 잡은 뒤 도입한다.

2. Replicator augmentation (커스텀 노이즈)

RTX stereo 모델이 커버하지 못하는 노이즈 수식을 직접 넣을 때 쓴다. warp 커널이 GPU에서 어노테이터 출력을 그 자리에서 변형한다.

필수: script node opt-in

이걸 빼면 조용히 실패한다. augmentation은 OmniGraph script node 위에서 돌고, 보안상 opt-in이 필요하다. 켜지 않으면 augmented annotator가 빈 배열 (0,) 을 반환하고 로그에만 경고가 남는다.

```
[Warning] [omni.replicator.core.ogn.python.impl.nodes.OgnAugment]
Augmentation cannot run, script nodes are disabled.
```

실행 인자로 켜다.

```
--/app/omni.graph.scriptnode/opt_in=true
```

사용법

```
import warp as wp
import omni.replicator.core as rep

@wp.kernel
def axial_noise(data_in: wp.array2d(dtype=wp.float32),
               data_out: wp.array2d(dtype=wp.float32),
               seed: int):
    """Kinect식 축방향 노이즈: sigma가 거리 제곱에 비례."""
    i, j = wp.tid()
    state = wp.rand_init(seed, i * data_in.shape[1] + j)
    z = data_in[i, j]
    data_out[i, j] = z + wp.randn(state) * (0.0012 * z * z)

rp = rep.create.render_product("/World/cam", (640, 480)) # 여기는 (width, height)
annotator = rep.annotators.get("distance_to_image_plane").augment(
    rep.annotators.Augmentation.from_function(axial_noise)
)
annotator.attach(rp)
depth = annotator.get_data().numpy()
```

여러 개를 연결하려면 `augment_compose([aug1, aug2])` 를 쓴다.

커널 시그니처에서 주의할 점:

- `data_in` / `data_out` 은 필수, `seed` 는 선택. `seed`를 받으면 Replicator가 전역 시드와 노드 id로 재현 가능한 값을 넣어 준다.
- 배열 `rank`와 `dtype`이 어노테이터 출력과 정확히 일치해야 한다. replicator render product의 `distance_to_image_plane` 은 2-D (H, W) 이고, 위 sensor API로 받는 배열은 3-D (H, W, 1) 이다. `rank`를 틀리면 역시 빈 배열이 나온다.
- 출력 shape이 입력과 다르면 `from_function(..., data_out_shape=...)` 을 준다.

검증 결과

항목	측정	기대
측방향 노이즈 $\sigma=0.0012 \cdot z^2$ ($z=1.0 \sim 5.59$ m, 307,200 px)	0.011863 m	0.011886 m
dropout 5% 채이닝 (<code>augment_compose</code>)	5.03%	5.00%

모델식과 0.2% 이내로 일치한다.

3. 재현 방법

검증 스크립트는 이 저장소에 포함하지 않았다. 재현하려면 프로젝트 타이밍 계약 (240 Hz physics / 60 Hz render) 과 아래 인자로 Isaac Sim python을 직접 실행한다.

```
~/isaacsim/python.sh <script>.py \
--/app/omni.graph.scriptnode/opt_in=true \
--/app/runLoops/main/rateLimitEnabled=true \
--/app/runLoops/main/rateLimitFrequency=60 \
--/renderer/multiGpu/enabled=false \
--/plugins/carb.tasking.plugin/threadCount=8 \
--/plugins/omni.tbb.globalcontrol/maxThreadCount=8
```

데이터를 읽기 전에 `world.reset()` → `app_utils.play(commit=True)` → warm-up 60 프레임이 필요하다. 렌더 파이프라인이 안정되기 전에는 빈 배열이나 과도기 값이 나온다.

4. 이 워크스페이스에 적용할 때

- 현재 `source/sim/camera.py` 의 `WristCamera` 는 `add_pointcloud_to_frame()` 경로라 노이즈가 없다. depth sensor로 바꾸면 `get_pointcloud()` 의 반환 형태가 Nx3에서 (H, W, 3) 으로 바뀌므로 변환부를 함께 고쳐야 한다.
- point completion 학습 데이터를 만들 때는 이 노이즈가 핵심이다. Projected-ShapeNet의 partial은 이상적 depth 투영이라, 같은 물체를 이 stereo sensor로 렌더하면 실제 센서 분포에 훨씬 가까운 입력이 된다.
- 물리 상호작용까지 갈 때만 D455 프리셋의 스케일 문제를 해결하면 된다. 그전까지는 plain camera + 수동 파라미터로 충분하다.